

Министерство науки и высшего образования Российской
Федерации федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский
университет имени академика С.П. Королева»
Институт информатики и кибернетики

Отчет по лабораторной работе № 6

Дисциплина: «ООП»

Тема «Многопоточное приложение»

Выполнил: Сучков Борис Антонович

Группа: 6201-120303D

Самара 2025

Задание на лабораторную работу

Разработать приложение, формирующее в одном потоке вычислений набор заданий для интегрирования, а во втором потоке – вычисляющее значения интегралов функций.

Official disclaimer (официальное уведомление). Некоторые элементы дизайна, навязанные в задании, являются не очень корректными (например, вместо одного объекта задания, используемого разными нитями, в данном случае разумно использовать очередь заданий, и т.д.). Однако корректная реализация и её исследование по своей сложности выходят за рамки одной лабораторной работы.

Задание 1

Добавьте в класс `Functions` метод, возвращающий значение интеграла функции, вычисленное с помощью численного метода.

В качестве параметров метод должен получать ссылку типа `Function` на объект функции, значения левой и правой границы области интегрирования, а также шаг дискретизации.

Если интервал интегрирования выходит за границы области определения функции, метод должен выбрасывать исключение.

Вычисление значения интеграла должно выполняться по методу трапеций. Для этого вся область интегрирования разбивается на участки, длина которых (кроме одного) равна шагу дискретизации. На каждом таком участке площадь под кривой, описываемой заданной функцией, приближается площадью трапеции, две вершины которой расположены на оси абсцисс на границах участка, а ещё две – на кривой в точках границ участка. Обратите внимание на то, что в область интегрирования необязательно укладывается целое количество шагов дискретизации.

В методе `main()` проверьте работу метода интегрирования. Для этого вычислите интеграл для экспоненты на отрезке от 0 до 1. Определите также, какой шаг дискретизации нужен, чтобы рассчитанное значение отличалось от теоретического в 7 знаке после запятой.

Задание 2

Далее в приложении один поток инструкций будет генерировать задачи для интегрирования, а второй поток – решать их. Для этого потребуется объект задания, через который эти потоки будут взаимодействовать. В объекте будут храниться параметры задания.

Создайте пакет `threads`, в котором будут размещены классы, связанные с потоками.

В пакете `threads` опишите класс `Task`, объект которого должен хранить ссылку на объект интегрируемой функции, границы области интегрирования, шаг дискретизации, а также целочисленное поле, хранящее количество выполняемых заданий. Т.е. объект описывает одно задание.

В главном классе программы опишите метод `nonThread()`, реализующий последовательную (без применения потоков инструкций) версию программы. В методе необходимо создать объект класса `Task` и установить в нём количество выполняемых заданий (минимум 100). После этого в цикле (до количества заданий) нужно выполнить следующие действия:

1. создать и поместить в объект задания объект логарифмической функции, основание которой является случайной величиной, распределённой равномерно на отрезке от 1 до 10;
2. указать в объекте задания левую границу области интегрирования (случайно распределена на отрезке от 0 до 100);
3. указать в объекте задания правую границу области интегрирования (случайно распределена на отрезке от 100 до 200);
4. указать в объекте задания шаг дискретизации (случайно распределён на отрезке от 0 до 1);
5. вывести в консоль сообщение вида "Source <левая граница> <правая граница> <шаг дискретизации>";
6. вычислить значение интеграла для параметров из объекта задания; 7. вывести в консоль сообщение вида "Result <левая граница> <правая граница> <шаг дискретизации> <результат интегрирования>".

Проверьте работу метода, вызвав его в методе `main()`.

Задание 3

В пакете `threads` создайте два следующих класса. При их реализации воспользуйтесь фрагментами последовательной версии программы из предыдущего задания.

Класс `SimpleGenerator` должен реализовывать интерфейс `Runnable`, получать в конструкторе и сохранять в своё поле ссылку на объект типа `Task`, а в методе `run()` в цикле должны формироваться задачи и заносятся в полученный объект задания, а также выводиться сообщения в консоль.

Класс `SimpleIntegrator` должен реализовывать интерфейс `Runnable`, получать в конструкторе и сохранять в своё поле ссылку на объект типа `Task`, а в методе `run()` в цикле должны решаться задачи, данные для которых берутся из полученного объекта задания, а также выводиться сообщения в консоль.

В главном классе программы создайте метод `simpleThreads()`. В нём создайте объект задания, укажите количество выполняемых заданий (минимум 100), создайте и

запустите два потока вычислений, основанных на описанных классах `SimpleGenerator` и `SimpleIntegrator`. Проверьте работу метода, вызвав его в методе `main()`.

Попробуйте запускать программу (несколько раз). Попробуйте запускать программу, изменяя приоритеты потоков перед их запуском. Убедитесь, что в некоторых случаях интегрирующий поток может прекращать своё выполнение из-за исключения `NullPointerException`. Определите причину возникновения этого исключения и сделайте так, чтобы оно не возникало (используйте какое-нибудь простое решение, без синхронизации и нотификаций). Попробуйте запускать программу (несколько раз). Убедитесь, что в некоторых случаях интегрирующий поток выводит сообщение с набором данных, который не встречается в сообщениях генерирующего потока (например, левая граница области интегрирования оказывается взята из одного задания, а правая граница и шаг дискретизации – из другого). Определите причину возникновения таких ситуаций и устраните её, используя блоки синхронизации в методах `run()` генерирующего и интегрирующего потоков. Убедитесь, что в коде нигде не возникает необоснованно длительной блокировки объектов.

Задание 4

Определите причину того, что не все сгенерированные задания оказываются выполнены интегрирующим потоком.

Для устранения этой проблемы потребуется дополнительный объект, представляющий собой одноместный семафор, различающий операции чтения и записи в защищаемый объект (пример класса таких объектов был приведён в лекции).

В пакете `threads` создайте два следующих класса. При их реализации воспользуйтесь фрагментами классов из предыдущего задания.

Класс `Generator` должен расширять класс `Thread`, получать в конструкторе и сохранять в свои поля ссылки на объект типа `Task` и на объект семафора, а в методе `run()` должны выполняться те же действия, что и в предыдущей версии генерирующего класса.

Класс `Integrator` должен расширять класс `Thread`, получать в конструкторе и сохранять в свои поля ссылки на объект типа `Task` и на объект семафора, а в методе `run()` должны выполняться те же действия, что и в предыдущей версии интегрирующего класса.

Отличие этих классов от предыдущих версий должно заключаться в том, что вместо средств синхронизации в методах `run()` должны использоваться возможности семафора.

В главном классе программы создайте метод `complicatedThreads()`. В нём создайте объект задания, укажите количество выполняемых заданий (минимум 100), создайте

и запустите два потока вычислений классов `Generator` и `Integrator`. Проверьте работу метода, вызвав его в методе `main()`.

Попробуйте запускать программу (несколько раз). Попробуйте запускать программу, изменяя приоритеты потоков перед их запуском. Сделайте так, чтобы после создания ваших потоков основной поток программы выжидал 50 миллисекунд, после чего прерывал работу ваших потоков путём вызова метода `interrupt()`. Измените код ваших классов потоков так, чтобы прерывание происходило и происходило корректно.

Задание 1

Я добавил в класс `Functions` метод, возвращающий значение интеграла функции, вычисленное с помощью численного метода.

В качестве параметров метод получает ссылку типа `Function` на объект функции, значения левой и правой границы области интегрирования, а также шаг дискретизации.

Если интервал интегрирования выходит за границы области определения функции, метод выбрасывает исключение.

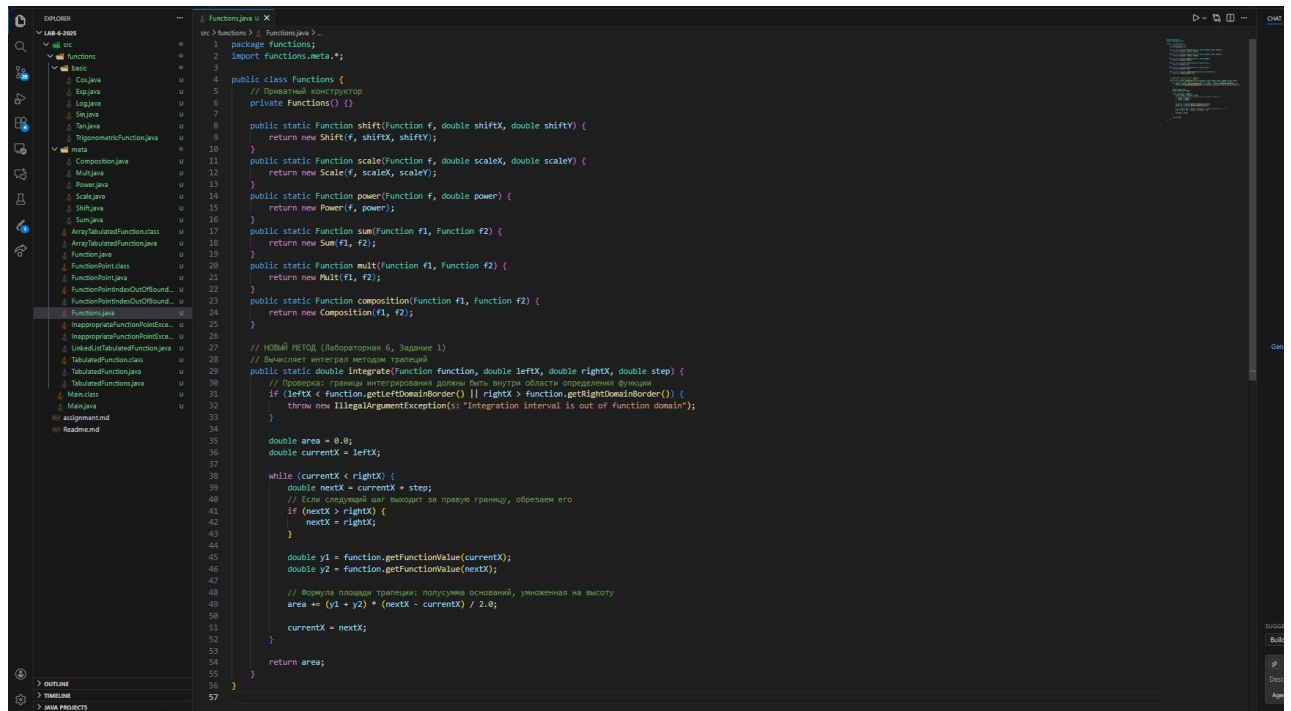
Вычисление значения интеграла выполняется по методу трапеций. Для этого вся область интегрирования разбивается на участки, длина которых (кроме одного) равна шагу дискретизации. На каждом таком участке площадь под кривой, описываемой заданной функцией, приближается площадью трапеции, две вершины которой расположены на оси абсцисс на границах участка, а ещё две – на кривой в точках границ участка. В область интегрирования необязательно укладывается целое количество шагов дискретизации.

В методе `main()` я проверил работу метода интегрирования. Для этого я вычислил интеграл для экспоненты на отрезке от 0 до 1. Определил также, какой шаг дискретизации нужен, чтобы рассчитанное значение отличалось от теоретического в 7 знаке после запятой.

Чтобы достичь точности 10^{-7} (седьмой знак после запятой) при интегрировании экспоненты на отрезке $[0, 1]$ методом трапеций, шаг дискретизации должен быть равен 0.000390625.

Этот результат получается путем последовательного деления начального шага 0.1 на 2^8 раз:

$0.1 / 2^8 = 0.000390625$. При этом шаге погрешность составляет примерно $3.4 \cdot 10^{-8}$, что удовлетворяет условию задачи (меньше 10^{-7}).



Задание 2

Далее в приложении один поток инструкций генерирует задачи для интегрирования, а второй поток – решает их. Для этого требуется объект задания, через который эти потоки будут взаимодействовать. В объекте хранится параметры задания.

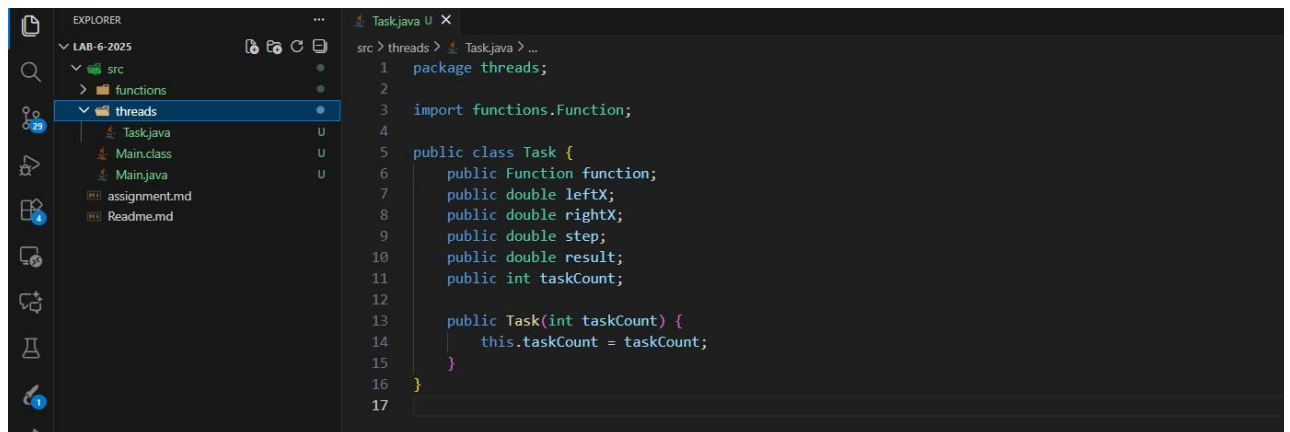
Я создайте пакет `threads`, в котором размещены классы, связанные с потоками. В пакете `threads` я описал класс `Task`, объект которого хранит ссылку на объект интегрируемой функции, границы области интегрирования, шаг дискретизации, а также целочисленное поле, хранящее количество выполняемых заданий. Т.е. объект описывает одно задание.

В главном классе программы я описал метод `nonThread()`, реализующий последовательную (без применения потоков инструкций) версию программы. В методе я создал объект класса `Task` и установил в нём количество выполняемых заданий (минимум 100). После этого в цикле (до количества заданий) я выполнил следующие действия:

1. Я создал и поместил в объект задания объект логарифмической функции, основание которой является случайной величиной, распределённой равномерно на отрезке от 1 до 10;
2. Указал в объекте задания левую границу области интегрирования (случайно распределена на отрезке от 0 до 100);
3. Указал в объекте задания правую границу области интегрирования (случайно распределена на отрезке от 100 до 200);

4. Указал в объекте задания шаг дискретизации (случайно распределён на отрезке от 0 до 1);
5. Вывел в консоль сообщение вида "Source <левая граница> <правая граница> <шаг дискретизации>";
6. Вычислил значение интеграла для параметров из объекта задания;
7. Вывел в консоль сообщение вида "Result <левая граница> <правая граница> <шаг дискретизации> <результат интегрирования>".

Я проверьте работу метода, вызвав его в методе main().



Задание 3

В пакете threads я создал два следующих класса. При их реализации я воспользовался фрагментами последовательной версии программы из предыдущего задания.

Класс SimpleGenerator реализует интерфейс Runnable, получает в конструкторе и сохраняет в своё поле ссылку на объект типа Task, а в методе run() в цикле формируются задачи и заносятся в полученный объект задания, а также выводятся сообщения в консоль.

Класс SimpleIntegrator реализует интерфейс Runnable, получает в конструкторе и сохраняет в своё поле ссылку на объект типа Task, а в методе run() в цикле решаются задачи, данные для которых берутся из полученного объекта задания, а также выводятся сообщения в консоль.

В главном классе программы я создал метод simpleThreads(). В нём создал объект задания, указал количество выполняемых заданий (минимум 100), создал и запустил два потока вычислений, основанных на описанных классах SimpleGenerator и SimpleIntegrator. Я проверил работу метода, вызвав его в методе main().

Я попробовал запускать программу (несколько раз). Попробовал запускать программу, изменяя приоритеты потоков перед их запуском. Убедился, что в некоторых случаях интегрирующий поток может прекращать своё выполнение из-за исключения NullPointerException. Определил причину возникновения этого исключения и сделал так, чтобы оно не возникало (использовал простое решение,

без синхронизации и нотификаций). Я попробовал запускать программу (несколько раз). Убедился, что в некоторых случаях интегрирующий поток выводит сообщение с набором данных, который не встречается в сообщениях генерирующего потока (например, левая граница области интегрирования оказывается взята из одного задания, а правая граница и шаг дискретизации – из другого). Определил причину возникновения таких ситуаций и устранил её, используя блоки синхронизации в методах `run()` генерирующего и интегрирующего потоков. Убедился, что в коде нигде не возникает необоснованно длительной блокировки объектов.

```

1 package threads;
2
3 import functions.basic.Log;
4 import java.util.Random;
5
6 public class SimpleGenerator implements Runnable {
7     private final Task task;
8
9     public SimpleGenerator(Task task) {
10         this.task = task;
11     }
12
13     public void run() {
14         Random random = new Random();
15         for (int i = 0; i < task.taskCount; i++) {
16             synchronized (task) {
17                 task.function = new Log(1 + (random.nextDouble() * 9));
18                 task.leftX = random.nextDouble() * 100;
19                 task.rightX = 100 + random.nextDouble() * 100;
20                 task.step = random.nextDouble();
21
22                 System.out.println("Source: " + task.leftX + " " + task.rightX + " " + task.step);
23             }
24             try {
25                 Thread.sleep(10);
26             } catch (InterruptedException e) {
27                 Thread.currentThread().interrupt();
28                 break;
29             }
30         }
31     }
32 }
33

```

```

1 package threads;
2
3 import functions.Functions;
4
5 public class SimpleIntegrator implements Runnable {
6     private final Task task;
7
8     public SimpleIntegrator(Task task) {
9         this.task = task;
10     }
11
12     public void run() {
13         for (int i = 0; i < task.taskCount; i++) {
14             synchronized (task) {
15                 if (task.function == null) {
16                     // Если данных нет, уменьшаем счетчик, чтобы остаться на этой же итерации и попробовать снова
17                     i--;
18                     continue;
19                 }
20                 double result = Functions.integrate(task.function, task.leftX, task.rightX, task.step);
21                 System.out.println("Result: " + task.leftX + " " + task.rightX + " " + task.step + " = " + result);
22
23                 // Зануляем функцию, чтобы не считать одно и то же дважды
24                 task.function = null;
25             }
26             try {
27                 Thread.sleep(10);
28             } catch (InterruptedException e) {
29                 Thread.currentThread().interrupt();
30                 break;
31             }
32         }
33     }
34 }
35

```

Задание 4

Я определите причину того, что не все сгенерированные задания оказываются выполнены интегрирующим потоком.

Для устранения этой проблемы потребуется дополнительный объект, представляющий собой одноместный семафор, различающий операции чтения и записи в защищаемый объект (пример класса таких объектов был приведён в лекции).

В пакете threads я создал два следующих класса. При их реализации я воспользовался фрагментами классов из предыдущего задания.

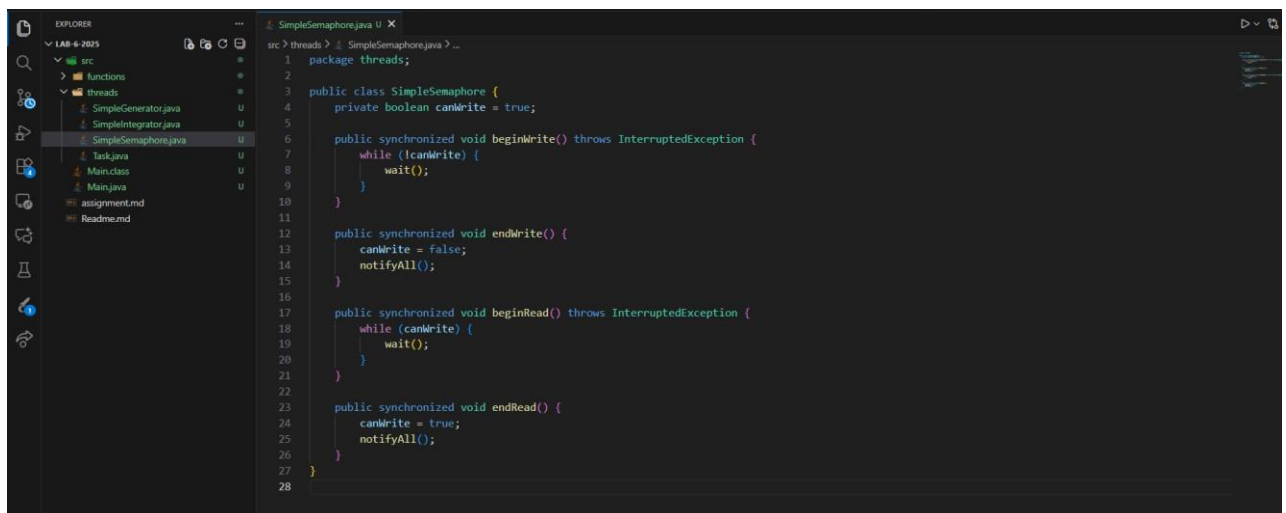
Класс Generator расширяет класс Thread, получает в конструкторе и сохраняет в свои поля ссылки на объект типа Task и на объект семафора, а в методе run() выполняются те же действия, что и в предыдущей версии генерирующего класса.

Класс Integrator расширяют класс Thread, получают в конструкторе и сохраняют в свои поля ссылки на объект типа Task и на объект семафора, а в методе run() выполняются те же действия, что и в предыдущей версии интегрирующего класса.

Отличие этих классов от предыдущих версий заключается в том, что вместо средств синхронизации в методах run() используются возможности семафора.

В главном классе программы я создал метод complicatedThreads(). В нём я создал объект задания, указал количество выполняемых заданий (минимум 100), создал и запустил два потока вычислений классов Generator и Integrator. Я проверил работу метода, вызвав его в методе main().

Я попробовал запускать программу (несколько раз). Попробовал запускать программу, изменяя приоритеты потоков перед их запуском. Сделал так, чтобы после создания ваших потоков основной поток программы выжидал 50 миллисекунд, после чего прерывал работу моих потоков путём вызова метода interrupt(). Изменил код ваших классов потоков так, чтобы прерывание происходило и происходило корректно.



```
1 package threads;
2
3 public class SimpleSemaphore {
4     private boolean canWrite = true;
5
6     public synchronized void beginWrite() throws InterruptedException {
7         while (!canWrite) {
8             wait();
9         }
10    }
11
12    public synchronized void endWrite() {
13        canWrite = false;
14        notifyAll();
15    }
16
17    public synchronized void beginRead() throws InterruptedException {
18        while (canWrite) {
19            wait();
20        }
21    }
22
23    public synchronized void endRead() {
24        canWrite = true;
25        notifyAll();
26    }
27 }
28
```

```
6 public class Generator extends Thread {
7     private final Task task;
8     private final SimpleSemaphore semaphore;
9
10    public Generator(Task task, SimpleSemaphore semaphore) {
11        this.task = task;
12        this.semaphore = semaphore;
13    }
14
15    @Override
16    public void run() {
17        Random random = new Random();
18        while (task.taskCount > 0 && !isInterrupted()) {
19            try {
20                semaphore.beginWrite();
21
22                task.function = new Log(1 + (random.nextDouble() * 9));
23                task.leftX = random.nextDouble() * 100;
24                task.rightX = 100 + random.nextDouble() * 100;
25                task.step = random.nextDouble();
26
27                task.taskCount--;
28
29                System.out.println("Source: " + task.leftX + " " + task.rightX + " " + task.step);
30
31                semaphore.endWrite();
32            } catch (InterruptedException e) {
33                interrupt();
34            }
35        }
36    }
37 }
38
39
```

```
3 import functions.Functions;
4
5 public class Integrator extends Thread {
6     private final Task task;
7     private final SimpleSemaphore semaphore;
8
9     public Integrator(Task task, SimpleSemaphore semaphore) {
10        this.task = task;
11        this.semaphore = semaphore;
12    }
13
14    public void run() {
15        while (!isInterrupted() && task.taskCount >= 0) {
16            try {
17                semaphore.beginRead();
18
19                if (task.function == null) {
20                    semaphore.endRead();
21                    break;
22                }
23
24                double result = Functions.integrate(task.function, task.leftX, task.rightX, task.step);
25                System.out.println("Result: " + task.leftX + " " + task.rightX + " " + task.step + " = " + result);
26
27                semaphore.endRead();
28            } catch (InterruptedException e) {
29                interrupt();
30            }
31        }
32    }
33 }
34
35
```

```
src > | Main.java > Main > simpleThreads()
1 import functions.*;
2 import functions.basic.Exp;
3 import functions.basic.Log;
4 import threads.*;
5 import java.util.Random;
6
7 public class Main {
8     public static void main(String[] args) {
9
10         // --- Задание 1: Интеграл ---
11         System.out.println(x: "--- Задание 1: Интегрирование экспоненты ---");
12         Function exp = new Exp();
13         double res = Functions.integrate(exp, leftX: 0, rightX: 1, step: 0.1);
14         double theoretical = Math.E - 1;
15         System.out.println("Результат (шаг 0.1): " + res);
16         System.out.println("Точное значение: " + theoretical);
17
18         double step = 0.1;
19         while (Math.abs(Functions.integrate(exp, leftX: 0, rightX: 1, step) - theoretical) >= 1e-7) {
20             step /= 2;
21         }
22         System.out.println("Шаг для точности 1e-7: " + step);
23         System.out.println("Результат с этим шагом: " + Functions.integrate(exp, leftX: 0, rightX: 1, step));
24
25         // --- Задание 2: NonThread ---
26         System.out.println(x: "\n--- Задание 2: nonThread() ---");
27         nonThread();
28
29         // --- Задание 3: Simple Threads ---
30         System.out.println(x: "\n--- Задание 3: simpleThreads() ---");
31         simpleThreads();
32
33         // --- Задание 4: Complicated Threads (Semaphore) ---
34         System.out.println(x: "\n--- Задание 4: complicatedThreads() ---");
35         complicatedThreads();
36     }
37
38     public static void nonThread() {
39         Task task = new Task(taskCount: 100);
40         Random random = new Random();
41         for (int i = 0; i < task.taskCount; i++) {
42             task.function = new Log(1 + (random.nextDouble() * 9));
43             task.leftX = random.nextDouble() * 100;
44             task.rightX = 100 + random.nextDouble() * 100;
45             task.step = random.nextDouble();
46         }
47     }
48 }
```

```
src > | Main.java > Main > nonThread()
39 public static void nonThread() {
40     Task task = new Task(taskCount: 100);
41     Random random = new Random();
42     for (int i = 0; i < task.taskCount; i++) {
43         task.function = new Log(1 + (random.nextDouble() * 9));
44         task.leftX = random.nextDouble() * 100;
45         task.rightX = 100 + random.nextDouble() * 100;
46         task.step = random.nextDouble();
47     }
48     System.out.println("Source: " + task.leftX + " " + task.rightX + " " + task.step);
49     double r = Functions.integrate(task.function, task.leftX, task.rightX, task.step);
50     System.out.println("Result: " + task.leftX + " " + task.rightX + " " + task.step + " " + r);
51 }
52
53
54
55 public static void simpleThreads() {
56     Task task = new Task(taskCount: 100);
57     SimpleGenerator generator = new SimpleGenerator(task);
58     SimpleIntegrator integrator = new SimpleIntegrator(task);
59
60     Thread t1 = new Thread(generator);
61     Thread t2 = new Thread(integrator);
62
63     // Запускаем потоки ОДНОВРЕМЕННО, без задержек
64     t1.start();
65     t2.start();
66
67     try {
68         t1.join();
69         t2.join();
70     } catch (InterruptedException e) {
71         e.printStackTrace();
72     }
73 }
74
75 public static void complicatedThreads() {
76     Task task = new Task(taskCount: 100);
77     SimpleSemaphore semaphore = new SimpleSemaphore();
78
79     Generator generator = new Generator(task, semaphore);
80     Integrator integrator = new Integrator(task, semaphore);
81
82     generator.start();
83     integrator.start();
84
85     try {
86         Thread.sleep(1000);
87         generator.interrupt();
88         integrator.interrupt();
89     } catch (InterruptedException e) {
90         e.printStackTrace();
91     }
92 }
93 }
```

Вывод в Main.java

--- Задание 1: Интегрирование экспоненты ---
Результат (шаг 0.1): 1.7197134913893144
Точное значение: 1.718281828459045
Шаг для точности 1e-7: 7.8125E-4
Результат с этим шагом: 1.7182819158553764

--- Задание 2: nonThread() ---

Source: 10.84677627271271 166.44952993210967 0.8191720344438476
Result: 10.84677627271271 166.44952993210967 0.8191720344438476 297.4426399153872
Source: 60.905936647905946 189.48155854101037 0.3406059790310164
Result: 60.905936647905946 189.48155854101037 0.3406059790310164 1475.7656758382273
Source: 21.057436922881244 156.04798374836457 0.6996149772093629
Result: 21.057436922881244 156.04798374836457 0.6996149772093629 269.0856781361542
Source: 32.47142111154234 153.3632628475351 0.8369315678517052
Result: 32.47142111154234 153.3632628475351 0.8369315678517052 537.002190173194
Source: 18.57974877795997 141.99138067573313 0.6516114265914156
Result: 18.57974877795997 141.99138067573313 0.6516114265914156 533.0291626105627
Source: 30.21019751248353 157.275005556752 0.7755266472628694
Result: 30.21019751248353 157.275005556752 0.7755266472628694 434.9338428484541
Source: 13.396098794458155 140.75589689962555 0.6145454526693283
Result: 13.396098794458155 140.75589689962555 0.6145454526693283 351.69234323733707
Source: 86.97381425445727 156.20385346698782 0.207698288713845
Result: 86.97381425445727 156.20385346698782 0.207698288713845 162.82771447874282
Source: 28.89689596733205 135.59418689764104 0.21441964935387747
Result: 28.89689596733205 135.59418689764104 0.21441964935387747 217.09483854755857
Source: 35.13715873200939 169.22135467780038 0.1299936342609439
Result: 35.13715873200939 169.22135467780038 0.1299936342609439 336.56525440367415
Source: 33.109318202151286 101.10898146397811 0.6506204975873339
Result: 33.109318202151286 101.10898146397811 0.6506204975873339 143.787867732435
Source: 2.747243207834793 115.9742684354993 0.9620604852987571
Result: 2.747243207834793 115.9742684354993 0.9620604852987571 213.49105243506074
Source: 82.67628282615159 107.22733486484827 0.6096004228515605
Result: 82.67628282615159 107.22733486484827 0.6096004228515605 59.689763382687055
Source: 16.36753189597172 196.83374176962656 0.5221272979363024
Result: 16.36753189597172 196.83374176962656 0.5221272979363024 1292.0474389449091
Source: 23.726295709298572 137.34114149738332 0.7093825127561572
Result: 23.726295709298572 137.34114149738332 0.7093825127561572 376.76972340297516
Source: 67.25845621260979 163.96854677362958 0.12900918595200395
Result: 67.25845621260979 163.96854677362958 0.12900918595200395 202.4704622899387
Source: 18.774785165184106 180.3795129947339 0.22879426653529
Result: 18.774785165184106 180.3795129947339 0.22879426653529 523.4746197762673
Source: 22.534738706701052 155.608030008392 0.7095460144461402
Result: 22.534738706701052 155.608030008392 0.7095460144461402 343.34517136465973
Source: 93.79757634793808 181.57869947978594 0.125336588094352
Result: 93.79757634793808 181.57869947978594 0.125336588094352 243.11053467526446
Source: 87.94015102544091 190.76037819610013 0.040378790243975615
Result: 87.94015102544091 190.76037819610013 0.040378790243975615 312.6645178658802
Source: 24.617072826900067 161.7151034418822 0.5955942826763843
Result: 24.617072826900067 161.7151034418822 0.5955942826763843 443.39072826522175
Source: 53.88031525093727 146.65182560607815 0.5029708568513067
Result: 53.88031525093727 146.65182560607815 0.5029708568513067 243.8841468248788
Source: 6.741853954355759 139.56708856542755 0.6820942613388274
Result: 6.741853954355759 139.56708856542755 0.6820942613388274 263.57199302381105
Source: 81.35420153028281 166.15244976837926 0.9807798392453175
Result: 81.35420153028281 166.15244976837926 0.9807798392453175 204.04653510528036
Source: 77.80022118046995 117.37115806179719 0.6805179442785261
Result: 77.80022118046995 117.37115806179719 0.6805179442785261 97.579053778063
Source: 9.424923139710328 167.45413993857298 0.7774866749391212
Result: 9.424923139710328 167.45413993857298 0.7774866749391212 400.72081989688684
Source: 86.70839969554696 114.03814299360727 0.3511735978089807
Result: 86.70839969554696 114.03814299360727 0.3511735978089807 93.29511599245821
Source: 90.87047221929122 153.26641400677957 0.6851882204952653
Result: 90.87047221929122 153.26641400677957 0.6851882204952653 184.52742612345608

Source: 28.713960591246757 155.89503400235435 0.18863543130389782
Result: 28.713960591246757 155.89503400235435 0.18863543130389782 258.43502972869214
Source: 87.654999554031 141.79566156886966 0.5814592409353574
Result: 87.654999554031 141.79566156886966 0.5814592409353574 121.24170526482307
Source: 90.4551185118815 159.7180559502894 0.253455404977868
Result: 90.4551185118815 159.7180559502894 0.253455404977868 153.68392091694315
Source: 82.86128983346087 166.77347469677687 0.18419048728792287
Result: 82.86128983346087 166.77347469677687 0.18419048728792287 177.31767365050845
Source: 74.17653331735579 167.82264347444863 0.3198539266177114
Result: 74.17653331735579 167.82264347444863 0.3198539266177114 480.0151548091702
Source: 90.72503204028884 118.79966424240905 0.8431087665792016
Result: 90.72503204028884 118.79966424240905 0.8431087665792016 192.1582498255914
Source: 95.24221186928894 119.97591919888163 0.34329590222538175
Result: 95.24221186928894 119.97591919888163 0.34329590222538175 79.23152029911157
Source: 71.41152373422219 125.26560756588637 0.4968854487386746
Result: 71.41152373422219 125.26560756588637 0.4968854487386746 205.38614917035662
Source: 72.08617944007536 171.8973124864313 0.5018583112353879
Result: 72.08617944007536 171.8973124864313 0.5018583112353879 310.8592652614556
Source: 82.17544389925398 191.1235366984634 0.17473963509899038
Result: 82.17544389925398 191.1235366984634 0.17473963509899038 296.7422339382855
Source: 52.7414937596218 195.1129227112795 0.3152954359914978
Result: 52.7414937596218 195.1129227112795 0.3152954359914978 453.15584929221313
Source: 21.544287671983554 147.76740016487616 0.6714420827847973
Result: 21.544287671983554 147.76740016487616 0.6714420827847973 303.1710072329788
Source: 47.08826598411135 117.92928803041397 0.5957081471136793
Result: 47.08826598411135 117.92928803041397 0.5957081471136793 190.94318674385835
Source: 48.166125186411705 182.29092759203832 0.27462641514395436
Result: 48.166125186411705 182.29092759203832 0.27462641514395436 303.62231410135047
Source: 58.9613085401902 192.2921494791263 0.23372353090223952
Result: 58.9613085401902 192.2921494791263 0.23372353090223952 1208.7386650053702
Source: 80.96227859783892 144.38851021226085 0.6733135858557563
Result: 80.96227859783892 144.38851021226085 0.6733135858557563 142.32734490486055
Source: 37.258009308876126 196.29585870270995 0.07409080592469874
Result: 37.258009308876126 196.29585870270995 0.07409080592469874 610.1033382171328
Source: 12.225568345436388 153.1645316033252 0.4265560449132596
Result: 12.225568345436388 153.1645316033252 0.4265560449132596 279.2250016300411
Source: 90.18708712031355 107.10903032533929 0.3846002396154531
Result: 90.18708712031355 107.10903032533929 0.3846002396154531 45.57685110249638
Source: 90.30473240187266 151.11676847910672 0.6211077603788612
Result: 90.30473240187266 151.11676847910672 0.6211077603788612 207.2765237586367
Source: 31.798845160009513 104.08210142440068 0.6722308704727332
Result: 31.798845160009513 104.08210142440068 0.6722308704727332 188.5346167485306
Source: 53.515652433230564 177.55949214778843 0.5970263727468598
Result: 53.515652433230564 177.55949214778843 0.5970263727468598 2567.218466192179
Source: 54.93752435855308 186.80238048480646 0.850433649825364
Result: 54.93752435855308 186.80238048480646 0.850433649825364 329.2683917565183
Source: 21.92001073416496 158.75188821618764 0.8701136730650575
Result: 21.92001073416496 158.75188821618764 0.8701136730650575 298.55234574918717
Source: 49.75642328434253 117.86224755650412 0.011455905611381323
Result: 49.75642328434253 117.86224755650412 0.011455905611381323 140.14368099902262
Source: 78.61202005717426 136.99742441217705 0.6301008530124846
Result: 78.61202005717426 136.99742441217705 0.6301008530124846 151.25379516145907
Source: 11.936375435451897 125.10610065569799 0.30761370488810647
Result: 11.936375435451897 125.10610065569799 0.30761370488810647 238.12971017731869
Source: 97.15380475239482 185.97980273017527 0.44348444241382146
Result: 97.15380475239482 185.97980273017527 0.44348444241382146 229.08881742313247
Source: 9.748216922548814 123.12169150654556 0.6954182335825722

Result: 9.748216922548814 123.12169150654556 0.6954182335825722 235.96761251872093
Source: 96.80011333475235 184.60107734291307 0.039919568652117765
Result: 96.80011333475235 184.60107734291307 0.039919568652117765 991.783579949838
Source: 4.099886848001011 138.74147770134124 0.5464622263095181
Result: 4.099886848001011 138.74147770134124 0.5464622263095181 2039.5150788774392
Source: 32.15296399167897 146.5876363242706 0.31107641620747906
Result: 32.15296399167897 146.5876363242706 0.31107641620747906 223.06113057376032
Source: 40.70875984547082 150.31960084833713 0.3789732278675222
Result: 40.70875984547082 150.31960084833713 0.3789732278675222 265.3297055292312
Source: 48.88681597643735 128.12960105924316 0.9008664239257717
Result: 48.88681597643735 128.12960105924316 0.9008664239257717 548.7036882635125
Source: 58.53985234007718 175.5501274709691 0.21393924985390722
Result: 58.53985234007718 175.5501274709691 0.21393924985390722 270.0830160134861
Source: 62.184964512750085 160.81929891752108 0.015944433521145385
Result: 62.184964512750085 160.81929891752108 0.015944433521145385 204.3844250534896
Source: 83.65051614952729 167.59217489362382 0.3097137157677
Result: 83.65051614952729 167.59217489362382 0.3097137157677 261.66481982243914
Source: 39.527714157814806 182.15445619899918 0.33539269361707325
Result: 39.527714157814806 182.15445619899918 0.33539269361707325 431.60521935101394
Source: 2.5822751424007806 104.26754770184469 0.2790055188116013
Result: 2.5822751424007806 104.26754770184469 0.2790055188116013 251.72817559964005
Source: 94.47298876132601 160.8716886966049 0.14849356612905618
Result: 94.47298876132601 160.8716886966049 0.14849356612905618 181.77263621371847
Source: 34.44036716498251 180.0965062670544 0.8421606014745068
Result: 34.44036716498251 180.0965062670544 0.8421606014745068 382.66613429850065
Source: 23.070006728331784 139.37594806063163 0.09949665302918098
Result: 23.070006728331784 139.37594806063163 0.09949665302918098 582.6962865910099
Source: 45.80168942113592 135.937299629214 0.7206413355909523
Result: 45.80168942113592 135.937299629214 0.7206413355909523 243.60654307678024
Source: 39.54662831454085 140.53154675850755 0.16828689972303423
Result: 39.54662831454085 140.53154675850755 0.16828689972303423 208.8669361415384
Source: 77.5917221826003 185.99508199232255 0.42460985118870387
Result: 77.5917221826003 185.99508199232255 0.42460985118870387 517.6155646783651
Source: 92.9110178180105 111.08036840518105 0.7658596985529632
Result: 92.9110178180105 111.08036840518105 0.7658596985529632 37.34753393558858
Source: 78.07107164473338 126.9658309700738 0.9385216151726067
Result: 78.07107164473338 126.9658309700738 0.9385216151726067 142.23520424424584
Source: 71.10822367054118 198.95291382520062 0.07350963061457616
Result: 71.10822367054118 198.95291382520062 0.07350963061457616 328.8543798141382
Source: 61.07226268443694 139.0809099333742 0.5586356093215895
Result: 61.07226268443694 139.0809099333742 0.5586356093215895 192.92233155844025
Source: 24.37233791915945 186.7377814260321 0.004638504322022841
Result: 24.37233791915945 186.7377814260321 0.004638504322022841 339.8836615230626
Source: 33.627397748370825 113.81789185198024 0.38634713297545864
Result: 33.627397748370825 113.81789185198024 0.38634713297545864 2830.332040363127
Source: 64.4148150042152 128.57522553816818 0.41808950027229863
Result: 64.4148150042152 128.57522553816818 0.41808950027229863 352.1913882477773
Source: 62.127305922670615 168.04325941481116 0.23124967674592223
Result: 62.127305922670615 168.04325941481116 0.23124967674592223 305.57772425447814
Source: 21.089606109068782 162.7404203588378 0.4619134608237284
Result: 21.089606109068782 162.7404203588378 0.4619134608237284 663.8858876662033
Source: 71.15282001895814 129.35261446741478 0.9278321086953193
Result: 71.15282001895814 129.35261446741478 0.9278321086953193 212.92340318917536
Source: 63.17573145620289 197.18202770632865 0.17647333672674603
Result: 63.17573145620289 197.18202770632865 0.17647333672674603 302.41319831361574
Source: 94.11879157175687 102.37403613805796 0.319717164832077
Result: 94.11879157175687 102.37403613805796 0.319717164832077 19.179253923914345

Source: 14.917895415729078 190.28172676911032 0.7357935519284433
 Result: 14.917895415729078 190.28172676911032 0.7357935519284433 804.9806365359324
 Source: 19.871470406230397 150.8001812028425 0.41534798455526933
 Result: 19.871470406230397 150.8001812028425 0.41534798455526933 250.98299662410307
 Source: 24.96579734850387 174.10031672275912 0.9560599487117202
 Result: 24.96579734850387 174.10031672275912 0.9560599487117202 443.5808886728628
 Source: 4.379896669317207 154.91478021742125 0.469028377423498
 Result: 4.379896669317207 154.91478021742125 0.469028377423498 358.45883606933376
 Source: 69.35273634430226 192.3759960096333 0.9501772158515362
 Result: 69.35273634430226 192.3759960096333 0.9501772158515362 304.04523691710205
 Source: 41.66188504452221 129.37914010205017 0.991223524627565
 Result: 41.66188504452221 129.37914010205017 0.991223524627565 232.08184093120585
 Source: 95.15416203220501 150.54049993481533 0.057289245882261475
 Result: 95.15416203220501 150.54049993481533 0.057289245882261475 128.48840676131
 Source: 13.085276610586261 111.04428510086775 0.9420270474113174
 Result: 13.085276610586261 111.04428510086775 0.9420270474113174 204.5420138656008
 Source: 36.466765898357565 100.01865194449614 0.42616962963243
 Result: 36.466765898357565 100.01865194449614 0.42616962963243 153.97720401644557
 Source: 59.185052220397495 151.55434605707973 0.6117010340614235
 Result: 59.185052220397495 151.55434605707973 0.6117010340614235 293.45327330085775
 Source: 59.34876603331507 104.0770020446796 0.8813875478563784
 Result: 59.34876603331507 104.0770020446796 0.8813875478563784 116.4953522125984
 Source: 43.136929961243375 174.5068485513912 0.9421947565447879
 Result: 43.136929961243375 174.5068485513912 0.9421947565447879 284.63738565352423
 Source: 17.36199390023406 157.4871606255041 0.9511562010638496
 Result: 17.36199390023406 157.4871606255041 0.9511562010638496 279.37089083386394
 Source: 0.028099432353900777 181.9355061200743 0.2935489738881666
 Result: 0.028099432353900777 181.9355061200743 0.2935489738881666 631.0315603730992
 Source: 24.35641145920987 157.269460170866 0.1857692832462009
 Result: 24.35641145920987 157.269460170866 0.1857692832462009 268.1922749345863

--- Задание 3: simpleThreads() ---

Source: 56.31060250784693 130.0659781344208 0.5302777981061376
 Result: 56.31060250784693 130.0659781344208 0.5302777981061376 = 153.14483945930692
 Source: 7.6678549315393525 146.00423941010013 0.5262703996784175
 Result: 7.6678549315393525 146.00423941010013 0.5262703996784175 = 1805.4267166558895
 Source: 30.95956641935168 173.78081472001446 0.3820921921667886
 Result: 30.95956641935168 173.78081472001446 0.3820921921667886 = 333.7805408746139
 Source: 45.66878155011932 126.93969828040159 0.30463642386700795
 Result: 45.66878155011932 126.93969828040159 0.30463642386700795 = 223.27989778865822
 Source: 98.04587720770276 168.60966334859688 0.3160224161850729
 Result: 98.04587720770276 168.60966334859688 0.3160224161850729 = 314.32844579593484
 Source: 96.08966745161035 154.91425519347362 0.8174420803817611
 Result: 96.08966745161035 154.91425519347362 0.8174420803817611 = 187.31658518719365
 Source: 1.6552379156504538 121.07139795021942 0.8311005208400729
 Result: 1.6552379156504538 121.07139795021942 0.8311005208400729 = 222.4680398915472
 Source: 74.711052317796 145.3663215269759 0.5662770450712755
 Result: 74.711052317796 145.3663215269759 0.5662770450712755 = 190.87541377243065
 Source: 4.709364904905666 170.02919228050644 0.4955724901437508
 Result: 4.709364904905666 170.02919228050644 0.4955724901437508 = 314.7787126581872
 Source: 57.10030729451563 180.02145382622984 0.6986972829561843
 Result: 57.10030729451563 180.02145382622984 0.6986972829561843 = 285.5721273496721
 Source: 9.211443571870815 140.83260408650906 0.8336943725616248
 Result: 9.211443571870815 140.83260408650906 0.8336943725616248 = 288.18092487709606
 Source: 49.03104553917553 174.6116981902719 0.7292798565062436
 Result: 49.03104553917553 174.6116981902719 0.7292798565062436 = 317.30420387935175
 Source: 62.87463847073167 110.3056213139256 0.25702282785868347

Result: 62.87463847073167 110.3056213139256 0.25702282785868347 = 100.23432065108423
 Source: 43.48089701262021 177.94873803825905 0.8383759469702478
 Result: 43.48089701262021 177.94873803825905 0.8383759469702478 = 909.7976111177803
 Source: 8.77428692426686 129.84627824845705 0.6691065862963996
 Result: 8.77428692426686 129.84627824845705 0.6691065862963996 = 291.1891508393005
 Source: 33.34827901715974 162.11638404486024 0.5024674622096057
 Result: 33.34827901715974 162.11638404486024 0.5024674622096057 = 419.62445736264266
 Source: 59.66441004560964 178.9034602874088 0.622485231125025
 Result: 59.66441004560964 178.9034602874088 0.622485231125025 = 272.9158779231744
 Source: 74.02878690449725 188.54810186798903 0.46003415793246694
 Result: 74.02878690449725 188.54810186798903 0.46003415793246694 = 279.72150192387966
 Source: 3.5991367237061755 146.0965372718024 0.156901063389546
 Result: 3.5991367237061755 146.0965372718024 0.156901063389546 = 279.0819761997035
 Source: 14.991819319016919 159.75516339616772 0.26683715986929324
 Result: 14.991819319016919 159.75516339616772 0.26683715986929324 = 284.8997993209909
 Source: 72.49969461227248 160.2434463102831 0.6976308250719333
 Result: 72.49969461227248 160.2434463102831 0.6976308250719333 = 229.62047521254922
 Source: 44.498094437673196 191.47105827929244 0.9445252996967023
 Result: 44.498094437673196 191.47105827929244 0.9445252996967023 = 316.3155474193189
 Source: 30.307589752026264 118.5628453986356 0.6428432258525782
 Result: 30.307589752026264 118.5628453986356 0.6428432258525782 = 200.9986591888438
 Source: 16.598635064096655 164.9777969186309 0.4764703534161574
 Result: 16.598635064096655 164.9777969186309 0.4764703534161574 = 571.2763790621324
 Source: 62.364571496525336 184.31595504090302 0.5188885056211661
 Result: 62.364571496525336 184.31595504090302 0.5188885056211661 = 551.7588281081879
 Source: 54.55589021402818 144.3797096423252 0.15473772165419797
 Result: 54.55589021402818 144.3797096423252 0.15473772165419797 = 178.88547517187413
 Source: 72.66005429027878 151.15373972330272 0.8324265666157483
 Result: 72.66005429027878 151.15373972330272 0.8324265666157483 = 358.8933333889293
 Source: 49.82759563661278 168.59039335880004 0.032098305314657294
 Result: 49.82759563661278 168.59039335880004 0.032098305314657294 = 315.4218029082836
 Source: 20.586578813359402 196.43502036990103 0.9982996018464557
 Result: 20.586578813359402 196.43502036990103 0.9982996018464557 = 456.82628018151826
 Source: 0.753996950239133 152.7984171198702 0.9829884662268542
 Result: 0.753996950239133 152.7984171198702 0.9829884662268542 = 409.6460867004455
 Source: 53.84395945426779 170.7811703762991 0.9720555972204243
 Result: 53.84395945426779 170.7811703762991 0.9720555972204243 = 270.8042164387027
 Source: 60.089078815649685 159.17740286821953 0.1273238783742645
 Result: 60.089078815649685 159.17740286821953 0.1273238783742645 = 205.32273831431976
 Source: 19.132738623237344 196.13299752616285 0.4660289991028257
 Result: 19.132738623237344 196.13299752616285 0.4660289991028257 = 550.9434261795598
 Source: 85.51640817878796 121.60612483854152 0.6210734068390211
 Result: 85.51640817878796 121.60612483854152 0.6210734068390211 = 149.2060415466923
 Source: 57.27757773139538 169.57546215167477 0.8778848129718622
 Result: 57.27757773139538 169.57546215167477 0.8778848129718622 = 232.9681341440994
 Source: 12.29021786977238 134.16424314195572 0.06749990038093645
 Result: 12.29021786977238 134.16424314195572 0.06749990038093645 = 283.87514237394333
 Source: 14.421197152331 195.19583309768257 0.3105322535392683
 Result: 14.421197152331 195.19583309768257 0.3105322535392683 = 371.6628419813742
 Source: 92.14235870703143 138.16928743584225 0.05297340148704144
 Result: 92.14235870703143 138.16928743584225 0.05297340148704144 = 127.35861907991898
 Source: 25.92949722665905 191.240911734055 0.31231421944475624
 Result: 25.92949722665905 191.240911734055 0.31231421944475624 = 546.9102447501996
 Source: 67.59575756885971 167.52054569678495 0.5612634884469849
 Result: 67.59575756885971 167.52054569678495 0.5612634884469849 = 246.72668531088337
 Source: 54.648500193278906 144.83896953444537 0.2163791226778805
 Result: 54.648500193278906 144.83896953444537 0.2163791226778805 = 253.68365083864614

Source: 75.82498052416564 137.11543302057316 0.2702169461942815
 Result: 75.82498052416564 137.11543302057316 0.2702169461942815 = 131.8812385674255
 Source: 90.15369816925975 111.55199295812776 0.20814081684453956
 Result: 90.15369816925975 111.55199295812776 0.20814081684453956 = 44.33169379096796
 Source: 1.2045496605414985 184.10402750444842 0.9847267577624331
 Result: 1.2045496605414985 184.10402750444842 0.9847267577624331 = 889.8124607467726
 Source: 40.63248506001227 182.02388040804857 0.49973335559726206
 Result: 40.63248506001227 182.02388040804857 0.49973335559726206 = 306.41369022652765
 Source: 20.685619406948664 196.74734143002405 0.7098990228778468
 Result: 20.685619406948664 196.74734143002405 0.7098990228778468 = 425.5059057083511
 Source: 74.04025528650394 162.99021107879116 0.2403665414133348
 Result: 74.04025528650394 162.99021107879116 0.2403665414133348 = 211.29857738791037
 Source: 11.141661734207165 190.64498574479953 0.11832372848334727
 Result: 11.141661734207165 190.64498574479953 0.11832372848334727 = 481.44985579205076
 Source: 90.17080772310089 144.91521482137586 0.041488601159147986
 Result: 90.17080772310089 144.91521482137586 0.041488601159147986 = 126.84944741896682
 Source: 12.823751874478994 117.09785641039977 0.415355985164788
 Result: 12.823751874478994 117.09785641039977 0.415355985164788 = 220.61542425979744
 Source: 30.281684671451327 153.52155762318733 0.2456623137032653
 Result: 30.281684671451327 153.52155762318733 0.2456623137032653 = 255.20932058141238
 Source: 68.06611958915376 113.37142685471467 0.856313482911206
 Result: 68.06611958915376 113.37142685471467 0.856313482911206 = 149.50606235840112
 Source: 55.91507606835455 107.47857853547418 0.31806066524627097
 Result: 55.91507606835455 107.47857853547418 0.31806066524627097 = 107.06767776895002
 Source: 79.38254496033895 129.62532799518996 0.41949701114902127
 Result: 79.38254496033895 129.62532799518996 0.41949701114902127 = 171.249221363862
 Source: 53.55488671553436 129.51460531269947 0.9894884395862895
 Result: 53.55488671553436 129.51460531269947 0.9894884395862895 = 440.1762393327081
 Source: 56.09427193100218 192.27480379573552 0.3889396400185343
 Result: 56.09427193100218 192.27480379573552 0.3889396400185343 = 663.6670366723141
 Source: 46.760631746989404 183.21894510811006 0.029836005252746656
 Result: 46.760631746989404 183.21894510811006 0.029836005252746656 = 292.3034965470432
 Source: 43.6300267646064 188.54966347479967 0.8507915344930944
 Result: 43.6300267646064 188.54966347479967 0.8507915344930944 = 412.0028154378436
 Source: 22.41873949538784 132.79922591857718 0.7808712522418468
 Result: 22.41873949538784 132.79922591857718 0.7808712522418468 = 251.6104890542235
 Source: 54.63878112167231 182.01639357793903 0.35724596105628537
 Result: 54.63878112167231 182.01639357793903 0.35724596105628537 = 319.46893705068624
 Source: 2.7519396391660678 148.05966649662503 0.8272764841787232
 Result: 2.7519396391660678 148.05966649662503 0.8272764841787232 = 292.59837447230024
 Source: 46.05589237265117 193.82901154435592 0.26191318286602283
 Result: 46.05589237265117 193.82901154435592 0.26191318286602283 = 524.4001243901275
 Source: 19.727303125811524 101.86592732589938 0.4348358833636071
 Result: 19.727303125811524 101.86592732589938 0.4348358833636071 = 265.7446909208765
 Source: 11.263233277393947 109.47429939381628 0.4805483619144534
 Result: 11.263233277393947 109.47429939381628 0.4805483619144534 = 404.3681920833608
 Source: 73.50361249804766 132.3621494352577 0.6346803321752302
 Result: 73.50361249804766 132.3621494352577 0.6346803321752302 = 242.82443875786765
 Source: 81.48392949449631 195.77707441428453 0.6810396649236488
 Result: 81.48392949449631 195.77707441428453 0.6810396649236488 = 383.14558762314556
 Source: 81.55865221520095 167.20582247186195 0.08249545415796777
 Result: 81.55865221520095 167.20582247186195 0.08249545415796777 = 189.44102724870694
 Source: 55.55237442027669 137.9310617159406 0.15656298205862984
 Result: 55.55237442027669 137.9310617159406 0.15656298205862984 = 288.60947753591665
 Source: 50.137052147926106 132.08610892502293 0.07159939749561428
 Result: 50.137052147926106 132.08610892502293 0.07159939749561428 = 187.08877440817474
 Source: 2.2000690401510314 131.32104713859854 0.8858091944273923

Result: 2.2000690401510314 131.32104713859854 0.8858091944273923 = 295.4423899867264
 Source: 24.6593326981564 190.79122047263553 0.32879641046281494
 Result: 24.6593326981564 190.79122047263553 0.32879641046281494 = 339.97862995701223
 Source: 3.9979898197737396 187.90191039592176 0.5341906900221121
 Result: 3.9979898197737396 187.90191039592176 0.5341906900221121 = 410.26630981554825
 Source: 0.7327243727523136 189.09049792887055 0.47191260489717435
 Result: 0.7327243727523136 189.09049792887055 0.47191260489717435 = 385.18352271331526
 Source: 37.09606137597342 141.9001655344473 0.3999329672136547
 Result: 37.09606137597342 141.9001655344473 0.3999329672136547 = 221.82847747366483
 Source: 2.404451240908534 199.69683323327064 0.25792009884409717
 Result: 2.404451240908534 199.69683323327064 0.25792009884409717 = 415.9068206402547
 Source: 5.900375115033551 164.60364062831093 0.5589577646832451
 Result: 5.900375115033551 164.60364062831093 0.5589577646832451 = 354.2143867228821
 Source: 59.298048868618636 190.57500970905303 5.557332517059921E-4
 Result: 59.298048868618636 190.57500970905303 5.557332517059921E-4 =
 305.05787810838183
 Source: 76.82878049723675 123.00490814146721 0.7124639964876787
 Result: 76.82878049723675 123.00490814146721 0.7124639964876787 = 122.61029750404445
 Source: 39.27337561287687 163.86163404849506 0.7863406954289218
 Result: 39.27337561287687 163.86163404849506 0.7863406954289218 = 260.8461787228576
 Source: 80.28546254807463 142.0641075640494 0.4911127150750918
 Result: 80.28546254807463 142.0641075640494 0.4911127150750918 = 409.4971988602418
 Source: 18.25092437676119 137.66225338954013 0.36790735590276236
 Result: 18.25092437676119 137.66225338954013 0.36790735590276236 = 1352.0584483792456
 Source: 74.9951479212611 155.35678293053547 0.21972216488083463
 Result: 74.9951479212611 155.35678293053547 0.21972216488083463 = 546.1657454439153
 Source: 93.84107061127733 167.99130635458485 0.8635392863094948
 Result: 93.84107061127733 167.99130635458485 0.8635392863094948 = 976.7976318196893
 Source: 93.83755238889316 153.87234329875227 0.3963795740035603
 Result: 93.83755238889316 153.87234329875227 0.3963795740035603 = 169.3848973427397
 Source: 74.41887484097892 133.1205140815715 0.6418311151329841
 Result: 74.41887484097892 133.1205140815715 0.6418311151329841 = 260.77646416002733
 Source: 15.32268896709702 167.7287844436027 0.38987548582075104
 Result: 15.32268896709702 167.7287844436027 0.38987548582075104 = 656.3067012911079
 Source: 0.2850598530238835 127.60134298201149 0.020374181474294484
 Result: 0.2850598530238835 127.60134298201149 0.020374181474294484 = 660.9996384921814
 Source: 29.84768457023256 140.56930890181332 0.10936724079089266
 Result: 29.84768457023256 140.56930890181332 0.10936724079089266 = 1896.671023502887
 Source: 78.55381286417918 138.22716835286323 0.6573742189827916
 Result: 78.55381286417918 138.22716835286323 0.6573742189827916 = 199.97514660299692
 Source: 94.87570577554229 181.40643717304437 0.6501776710718388
 Result: 94.87570577554229 181.40643717304437 0.6501776710718388 = 268.92927289931845
 Source: 38.40799927782111 199.2007594010203 0.7651076187286684
 Result: 38.40799927782111 199.2007594010203 0.7651076187286684 = 527.9557605449766
 Source: 98.17762743521492 103.67926733022838 0.16963522332180458
 Result: 98.17762743521492 103.67926733022838 0.16963522332180458 = 21.939362950280525
 Source: 38.36244654108185 148.77270959192577 0.5241221380686952
 Result: 38.36244654108185 148.77270959192577 0.5241221380686952 = 239.52265023161874
 Source: 73.27707973547838 186.2894885104658 0.4968619522921429
 Result: 73.27707973547838 186.2894885104658 0.4968619522921429 = 290.01171696964803
 Source: 82.17589738017676 134.61643809647563 0.3270885670322119
 Result: 82.17589738017676 134.61643809647563 0.3270885670322119 = 176.1180613463482
 Source: 36.2394813626602 116.38177946231137 0.2304154888196316
 Result: 36.2394813626602 116.38177946231137 0.2304154888196316 = 158.6315259158999
 Source: 25.05228868888867 140.20539891371385 0.7312095694564517
 Result: 25.05228868888867 140.20539891371385 0.7312095694564517 = 442.97700151769277
 Source: 37.9456966110045 198.63421773734834 0.32769113425478247

Result: 37.9456966110045 198.63421773734834 0.32769113425478247 = 413.9274729030819
Source: 76.77193821190843 129.45809140598575 0.2480918304186045
Result: 76.77193821190843 129.45809140598575 0.2480918304186045 = 119.06167838887886
Source: 50.99092930758978 140.82432250624456 0.43999069902396415
Result: 50.99092930758978 140.82432250624456 0.43999069902396415 = 202.31877918103663

--- Задание 4: complicatedThreads() ---

Source: 52.07990900472751 124.97382768863696 0.525075948687299
Result: 52.07990900472751 124.97382768863696 0.525075948687299 = 163.5077234428375
Source: 32.22891072188347 124.94478164593235 0.7733448878220698
Result: 32.22891072188347 124.94478164593235 0.7733448878220698 = 184.42569762716468
Source: 34.891298538800385 108.34833265545663 0.41023025611062713
Result: 34.891298538800385 108.34833265545663 0.41023025611062713 = 270.3302812031961
Source: 2.7002562931559337 120.06442673618585 0.28763387369381455
Result: 2.7002562931559337 120.06442673618585 0.28763387369381455 = 1120.4986102119487
Source: 69.85251444451963 120.8663994013209 0.7911321154916463
Result: 69.85251444451963 120.8663994013209 0.7911321154916463 = 136.94001631141387
Source: 65.41379316135576 115.16369015130279 0.8703986649189758
Result: 65.41379316135576 115.16369015130279 0.8703986649189758 = 165.44486309855483
Source: 49.17973301829493 139.11144917121163 0.8479560677015597
Result: 49.17973301829493 139.11144917121163 0.8479560677015597 = 506.09975368671314
Source: 41.56995664081321 147.05398841611913 0.824646187007365
Result: 41.56995664081321 147.05398841611913 0.824646187007365 = 213.704558080053
Source: 63.03684719003094 154.6791508683978 0.43087327571456413
Result: 63.03684719003094 154.6791508683978 0.43087327571456413 = 213.86497176215278
Source: 55.59952952790918 145.79579879966417 0.1345880081262435
Result: 55.59952952790918 145.79579879966417 0.1345880081262435 = 180.82506686717068
Source: 19.36739277233076 126.30686761576368 0.3101440527254552
Result: 19.36739277233076 126.30686761576368 0.3101440527254552 = 262.745365812854
Source: 60.296490967385765 113.73546102987805 0.7965739873052974
Result: 60.296490967385765 113.73546102987805 0.7965739873052974 = 522.3834667991961
Source: 11.925966480510175 183.57173336208547 0.7294940429553836
Result: 11.925966480510175 183.57173336208547 0.7294940429553836 = 446.3836017945464
Source: 63.470634004214 154.74341039195872 0.017762618751728465
Result: 63.470634004214 154.74341039195872 0.017762618751728465 = 199.32062337467983
Source: 99.39970939008637 185.6729336190163 0.9541803701637193
Result: 99.39970939008637 185.6729336190163 0.9541803701637193 = 199.99305948693564
Source: 31.510589003882316 169.6694395791624 0.09893454561825299
Result: 31.510589003882316 169.6694395791624 0.09893454561825299 = 322.4469364094006
Source: 83.51052374824967 163.81385717426576 0.7551539683011441
Result: 83.51052374824967 163.81385717426576 0.7551539683011441 = 230.9269669125082
Source: 70.24787559533678 159.0306317408931 0.06785519875145396
Result: 70.24787559533678 159.0306317408931 0.06785519875145396 = 184.61611314307638
Source: 98.98861692254954 100.69971149318584 0.020724327365541306
Result: 98.98861692254954 100.69971149318584 0.020724327365541306 = 7.960223751017909
Source: 8.512594242404726 198.9475762530078 0.37959329453360136
Result: 8.512594242404726 198.9475762530078 0.37959329453360136 = 719.7546019164815
Source: 53.30800389545865 125.66626343884303 0.9371330551603186
Result: 53.30800389545865 125.66626343884303 0.9371330551603186 = 153.58466742336012
Source: 34.771410645313175 181.34451338046355 0.2502008716739028
Result: 34.771410645313175 181.34451338046355 0.2502008716739028 = 465.95406877288883
Source: 79.07460604647153 128.95527641610437 0.42944209313596104
Result: 79.07460604647153 128.95527641610437 0.42944209313596104 = 132.61020725639025
Source: 62.23841502086936 134.42900967147818 0.44016156380939586
Result: 62.23841502086936 134.42900967147818 0.44016156380939586 = 205.5129585861353
Source: 86.95756049406639 123.65233760731422 0.8041435458079955
Result: 86.95756049406639 123.65233760731422 0.8041435458079955 = 80.1051509931043

Source: 22.849535539947 164.466808018157 0.5540131379487332
 Result: 22.849535539947 164.466808018157 0.5540131379487332 = 371.0001525044556
 Source: 32.05436805343184 108.80263373910216 0.7848965074467472
 Result: 32.05436805343184 108.80263373910216 0.7848965074467472 = 145.10388097936058
 Source: 15.491081265322727 184.8674564579308 0.1725505709359787
 Result: 15.491081265322727 184.8674564579308 0.1725505709359787 = 406.48165143216437
 Source: 31.580500787809594 112.22034297086697 0.3827979274884544
 Result: 31.580500787809594 112.22034297086697 0.3827979274884544 = 687.3940308077017
 Source: 58.87306779147461 123.59636337604832 0.3840159398368228
 Result: 58.87306779147461 123.59636337604832 0.3840159398368228 = 264.50132521976514
 Source: 96.77409646429687 116.50726169417965 0.29259324854041224
 Result: 96.77409646429687 116.50726169417965 0.29259324854041224 = 74.85432508813732
 Source: 24.437944887812503 124.92889584945522 0.5306814952818852
 Result: 24.437944887812503 124.92889584945522 0.5306814952818852 = 722.2291926749134
 Source: 66.90637355794647 196.79626901359956 0.7562332400315216
 Result: 66.90637355794647 196.79626901359956 0.7562332400315216 = 296.61865009622767
 Source: 49.69534730945556 186.56213079434173 0.23723190309686926
 Result: 49.69534730945556 186.56213079434173 0.23723190309686926 = 356.1546855425598
 Source: 58.28838144902141 123.7352842059912 0.0851833052732961
 Result: 58.28838144902141 123.7352842059912 0.0851833052732961 = 128.52944437575462
 Source: 57.37349627266557 198.09990918530406 0.03735212011178346
 Result: 57.37349627266557 198.09990918530406 0.03735212011178346 = 355.6834867007038
 Source: 66.16470232090661 110.22732067389428 0.2828914838170292
 Result: 66.16470232090661 110.22732067389428 0.2828914838170292 = 89.93696707828572
 Source: 30.77787499422516 171.92680375345918 0.16492704594118768
 Result: 30.77787499422516 171.92680375345918 0.16492704594118768 = 316.7344568186859
 Source: 82.38476187184396 105.44226263276974 0.45882144922391543
 Result: 82.38476187184396 105.44226263276974 0.45882144922391543 = 45.506431338152986
 Source: 20.74780504681194 182.48661988407815 0.32625934957245417
 Result: 20.74780504681194 182.48661988407815 0.32625934957245417 = 779.4905218422438
 Source: 16.74232884014817 164.61601796656447 0.9754565378693114
 Result: 16.74232884014817 164.61601796656447 0.9754565378693114 = 749.7623794737991
 Source: 84.60155661541222 154.98108109927028 0.5197114858941295
 Result: 84.60155661541222 154.98108109927028 0.5197114858941295 = 149.50536056983202
 Source: 34.87387461217324 148.15261314171505 0.5196813398659311
 Result: 34.87387461217324 148.15261314171505 0.5196813398659311 = 233.5155154272946
 Source: 82.46058736932858 169.50427488785877 0.2191783233531306
 Result: 82.46058736932858 169.50427488785877 0.2191783233531306 = 192.90451819462427
 Source: 50.606382598441314 128.5449335329226 0.5399470441893534
 Result: 50.606382598441314 128.5449335329226 0.5399470441893534 = 310.5814860153498
 Source: 81.34358848336704 171.62855422640453 0.48779662884086006
 Result: 81.34358848336704 171.62855422640453 0.48779662884086006 = 218.096399766072
 Source: 35.64524809982355 108.62965344413578 0.7406264953959285
 Result: 35.64524809982355 108.62965344413578 0.7406264953959285 = 446.671393636023
 Source: 42.42089899514505 105.45744298637119 0.8382975255231674
 Result: 42.42089899514505 105.45744298637119 0.8382975255231674 = 145.89931734777434
 Source: 34.49289187679743 179.69204651208662 0.5749657277111703
 Result: 34.49289187679743 179.69204651208662 0.5749657277111703 = 312.9109149107505
 Source: 91.81996628704805 110.16122641528526 0.9064083857810802
 Result: 91.81996628704805 110.16122641528526 0.9064083857810802 = 41.37523735580801
 Source: 97.48871100733466 172.93626771447435 0.9280377327130421
 Result: 97.48871100733466 172.93626771447435 0.9280377327130421 = 173.95976900661068
 Source: 0.3724503876208707 146.01083523658602 0.25371023772432866
 Result: 0.3724503876208707 146.01083523658602 0.25371023772432866 = 324.6035061276684
 Source: 69.86099397173454 182.83672539951039 0.4311049507167424
 Result: 69.86099397173454 182.83672539951039 0.4311049507167424 = 415.41821193077067
 Source: 22.693604432380056 140.92563526562103 0.8663568866742668

Result: 22.693604432380056 140.92563526562103 0.8663568866742668 = 360.4856377304004
 Source: 99.681425785131 165.8862068295502 0.09879683068126477
 Result: 99.681425785131 165.8862068295502 0.09879683068126477 = 247.1870379069259
 Source: 70.98455356574917 173.55076312397742 0.16606229946479067
 Result: 70.98455356574917 173.55076312397742 0.16606229946479067 = 302.8815336335885
 Source: 52.612795959885105 143.25633020698496 0.23050843465427773
 Result: 52.612795959885105 143.25633020698496 0.23050843465427773 = 181.88539405219692
 Source: 23.948102388929204 133.87962963535995 0.5997621623852135
 Result: 23.948102388929204 133.87962963535995 0.5997621623852135 = 288.1191112954974
 Source: 46.34055374504273 160.88599934110522 0.7674264986308249
 Result: 46.34055374504273 160.88599934110522 0.7674264986308249 = 229.63879654323674
 Source: 96.20868652988356 127.43721136863849 0.12435604885499063
 Result: 96.20868652988356 127.43721136863849 0.12435604885499063 = 75.53224076451141
 Source: 42.57883311082398 185.51344509998006 0.5465892760196752
 Result: 42.57883311082398 185.51344509998006 0.5465892760196752 = 403.7853478913359
 Source: 18.627114086468886 114.03975265633369 0.1529363833257349
 Result: 18.627114086468886 114.03975265633369 0.1529363833257349 = 406.6808847902398
 Source: 59.541542006433914 140.1675037290322 0.14338631482250896
 Result: 59.541542006433914 140.1675037290322 0.14338631482250896 = 196.8920221487738
 Source: 4.900686441859225 197.14652910154422 0.7789104810382228
 Result: 4.900686441859225 197.14652910154422 0.7789104810382228 = 1103.7915280786208
 Source: 92.76918678793538 107.6337437475009 0.24390424885060524
 Result: 92.76918678793538 107.6337437475009 0.24390424885060524 = 29.767328597184257
 Source: 4.83936282974623 104.8192524332746 0.3444220065920255
 Result: 4.83936282974623 104.8192524332746 0.3444220065920255 = 7197.881939816339
 Source: 57.202027089531214 155.41107436813746 0.7385253128777612
 Result: 57.202027089531214 155.41107436813746 0.7385253128777612 = 287.4752154985271
 Source: 69.7085648539855 128.30477442869025 0.6694777658770147
 Result: 69.7085648539855 128.30477442869025 0.6694777658770147 = 418.3846170630684
 Source: 88.39019105680343 160.67445832719582 0.9781842969756628
 Result: 88.39019105680343 160.67445832719582 0.9781842969756628 = 192.89917059308124
 Source: 80.02926224337176 109.25672117126398 0.5081865543039893
 Result: 80.02926224337176 109.25672117126398 0.5081865543039893 = 86.05576388156086
 Source: 65.73137974286945 138.75934791396085 0.8846905381717257
 Result: 65.73137974286945 138.75934791396085 0.8846905381717257 = 161.5532628873132
 Source: 11.225954672664805 191.6116445828251 0.048515908828445364
 Result: 11.225954672664805 191.6116445828251 0.048515908828445364 = 412.93244457154225
 Source: 45.6267898409449 103.68348869355944 0.8565212532714933
 Result: 45.6267898409449 103.68348869355944 0.8565212532714933 = 834.8456852636807
 Source: 37.51317826456136 179.13535900632135 0.5219732855390619
 Result: 37.51317826456136 179.13535900632135 0.5219732855390619 = 569.6725217245751
 Source: 17.445861947589226 114.22167618468492 0.6614919690700047
 Result: 17.445861947589226 114.22167618468492 0.6614919690700047 = 201.6449591352299
 Source: 99.41117023018668 164.65506095317403 0.2396177263717595
 Result: 99.41117023018668 164.65506095317403 0.2396177263717595 = 350.23054838292944
 Source: 20.190508117039553 184.04914351359287 0.757710987093569
 Result: 20.190508117039553 184.04914351359287 0.757710987093569 = 498.9689626836798
 Source: 85.2813031264658 121.99286030396054 0.6208083504563917
 Result: 85.2813031264658 121.99286030396054 0.6208083504563917 = 107.29822448672857
 Source: 82.74941174480624 159.46916009059143 0.7342236018434554
 Result: 82.74941174480624 159.46916009059143 0.7342236018434554 = 175.79290221024826
 Source: 2.2719110120767994 176.61636652484623 0.031096024229568586
 Result: 2.2719110120767994 176.61636652484623 0.031096024229568586 = 540.8941979985655
 Source: 75.63305428599317 193.2404858027727 0.812752555197521
 Result: 75.63305428599317 193.2404858027727 0.812752555197521 = 336.87216805680845
 Source: 37.06516442888812 196.0219276411188 0.49566412113757685
 Result: 37.06516442888812 196.0219276411188 0.49566412113757685 = 415.04860981443414

Source: 87.08006259157463 168.02031664856204 0.601940915327862
 Result: 87.08006259157463 168.02031664856204 0.601940915327862 = 266.5352154906593
 Source: 31.815568061808275 165.86616031484107 0.6047251199436707
 Result: 31.815568061808275 165.86616031484107 0.6047251199436707 = 446.42976381471163
 Source: 61.42553658349814 147.9383163889417 0.5455037510317999
 Result: 61.42553658349814 147.9383163889417 0.5455037510317999 = 258.86065723237687
 Source: 25.670753650169196 142.86140527582046 0.1772787512980195
 Result: 25.670753650169196 142.86140527582046 0.1772787512980195 = 267.73533465251865
 Source: 62.62196222771574 161.8173842970297 0.7556945141628096
 Result: 62.62196222771574 161.8173842970297 0.7556945141628096 = 237.07431545489547
 Source: 37.265169570719834 100.4507263363433 0.4996384490351672
 Result: 37.265169570719834 100.4507263363433 0.4996384490351672 = 155.7110397262419
 Source: 50.803319607123576 146.55678412824972 0.17349289593947037
 Result: 50.803319607123576 146.55678412824972 0.17349289593947037 = 190.91184463308892